

并行整体刚度矩阵组装：符号阶段、正确性与内存证据

2026-05-22 例会汇报

Jiang Haohua

2026-05-22

我这次集中回答三个问题

① 符号组装阶段能不能并行，是否值得并行？

我比较同一数值后端下的串行符号构建和并行符号构建，重点看总耗时、临时内存和独立进程峰值内存。

② 正确性是不是每轮都验证？

我当前采用 C++ `cpu_serial` 组装矩阵作为正式参考矩阵；Abaqus/MATLAB 外部参考矩阵还没有纳入已完成证据。

③ 内存占用到底差在哪里？

我把“保留下来的结构内存”和“临时生成的贡献缓冲”分开解释，重点说明 **2.39 GiB** 的含义。

我的结论

我现在的阶段性判断是：符号组装已经并行化；20 物理线程下值得做；正确性目前按 C++ 串行参考矩阵严格比对。

我先把关键术语翻译成项目语境

先决定“哪里可能非零”

- **符号组装** (symbolic assembly): 先找全局矩阵的非零位置。
- **压缩稀疏行格式** (CSR): 按行存储非零列, 不存完整大矩阵。
- **写入计划** (scatter plan): 提前记住单元矩阵条目写回全局矩阵的位置。

再计算“这些位置的数值”

- **数值组装** (numeric assembly): 计算每个单元刚度矩阵。
- 单元矩阵条目按写入计划累加到全局矩阵。
- 网格拓扑不变时, 我可以复用符号结果, 只重复数值计算。

我的结论

符号组装像先画路网; 数值组装像沿着这套路网反复运输数值。

我固定了实验口径，避免混用不同平台的数字

算例与核函数

- 工程网格：WindHub
- 单元类型：四节点四面体 Tet4
- 物理核函数：physics_tet4
- 节点数：228384
- 单元数：1113684
- 自由度数：685152

平台与线程

- CPU：Intel Core Ultra 7 265KF
- 系统：Linux x86_64
- 编译器：GCC 13.3 + OpenMP
- 物理/逻辑核心：20/20
- 线程范围：1 到 20
- 主线数值后端：cpu_atomic

我的结论

下面的核心图表都来自同一组 Linux Intel 全核心结果，不把 Apple 平台的绝对时间混进来。

数据来源：[results/2026-05-20-linux-intel-symbolic-memory-full-host/linux_intel_symbolic_memory_report.md](https://github.com/robertodonato/2026-05-20-linux-intel-symbolic-memory-full-host/linux_intel_symbolic_memory_report.md)

符号组装已经并行，20 线程下总时间明显缩短

Symbolic build comparison with integer thread ticks

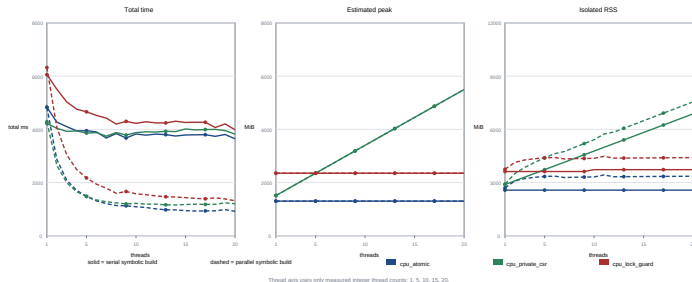


图 1 | 串行符号构建与并行符号构建的总体比较

横轴是实测整数线程数，只标出 1、5、10、15、20 这些整数刻度；纵轴分别表示总耗时、估算峰值内存和独立进程峰值内存。实线表示串行符号构建，虚线表示并行符号构建；颜色对应不同数值后端。

数据来源：Linux Intel 全核心符号内存报告；同一后端、同一线程数对比

在主线后端上，并行符号构建用内存换时间

cpu_atomic: serial symbolic build vs parallel symbolic build

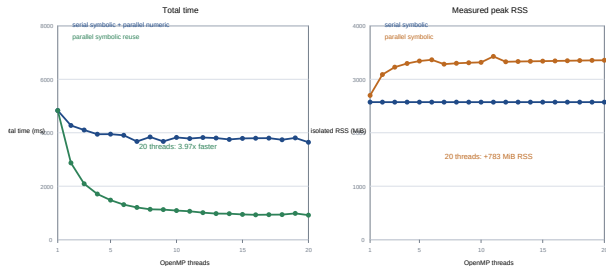


图 2 | 主线数值后端下的耗时与峰值内存

左图比较每次完整组装路径的总耗时，右图比较独立进程峰值常驻内存。横轴只显示实测整数线程点；蓝线是串行符号构建，绿/橙线是并行符号构建。20 线程时，总时间从 **3644 ms** 降到 **917 ms**。

数据来源：由 `isolated_symbolic_memory.csv` 派生；没有重跑基准测试

我把“为什么 1 线程不值得”单独列出来

线程数	串行符号构建总耗时 ms	并行符号构建总耗时 ms	加速比	并行符号构建峰值内存 MiB
1	4831.683	4830.872	1.00x	2698.7
4	3944.694	1707.861	2.31x	3295.7
8	3841.637	1135.618	3.38x	3299.4
20	3644.206	917.095	3.97x	3356.0

图 3 | 主线后端下的代表性线程点

表中总耗时包括符号构建和数值组装。1 线程时并行符号构建几乎不带来收益；线程数增加后，符号阶段可以分摊到多个核心上，20 线程时达到约 **3.97x** 的总时间改善。

数据来源：isolated_symbolic_memory.csv；数值后端为 cpu_atomic

正确性：我当前每轮都用 C++ 串行矩阵作参考

Correctness summary: current reference policy

evidence family	max relative L2	max absolute error	reference
symbolic/direct evaluation	1.618e-16	0.007812	C++ cpu_serial matrix
backend tradeoff sweep	1.517e-16	0.007812	C++ cpu_serial matrix
external Abaqus/MATLAB matrix	not used	not used	future cross-check

All current correctness rows compare candidate matrices against the C++ cpu_serial assembled matrix.

图 4 | 当前正确性参考口径

表中前两行分别汇总符号/直接组装评估和数值后端对比评估。误差指标都来自候选矩阵与 C++ `cpu_serial` 矩阵的比较；Abaqus/MATLAB 外部参考矩阵目前列为后续交叉验证，而不是已完成证据。

数据来源：由 isolated symbolic CSV 和 backend tradeoff CSV 派生；均为当前仓库已有结果

已有正确性热力图仍然用于多算法横向检查

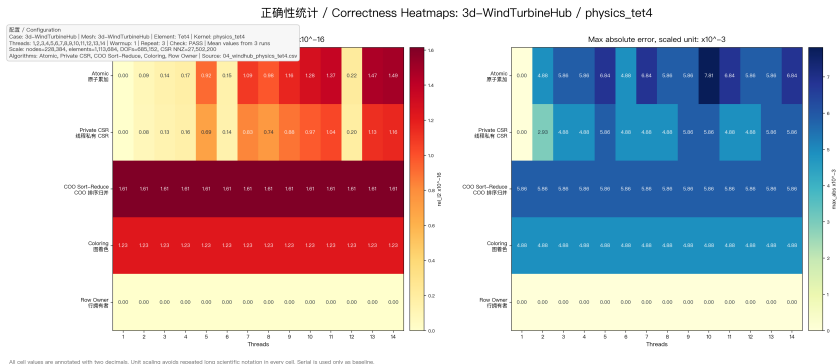


图 5 | WindHub 物理核函数的多算法正确性热力图

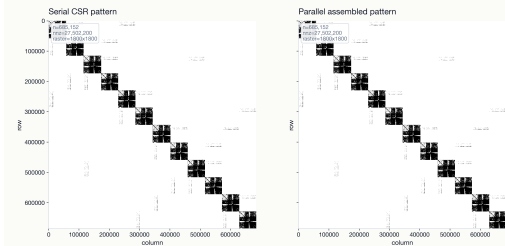
这张图来自早期 12 图结果包，用来横向检查不同算法、不同线程数是否通过正确性门槛。颜色对应误差水平；读图重点不是速度，而是先确认候选算法没有偏离 C++ 串行参考矩阵。

数据来源: results/2026-04-28-12charts-repeat3-threads1to14/presentation_charts_12_v2

Python 和 MATLAB 现在采用同一矩阵显示约定

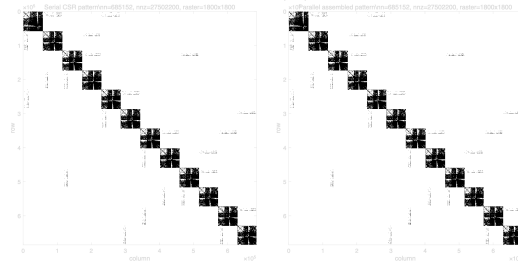
WindHub stiffness sparse pattern

3d-WindTurbineHub | physics_tst4 | cpu_stormc @ 14T | rel_D=1.502e-16, max_abs=6.839e-03



Python：行号向下、列号向右、非零元为黑点

WindHub stiffness sparse pattern



MATLAB：同一坐标约定和灰度约定

图 6 | 同一矩阵表示方式下的 Python/MATLAB 可视化核对

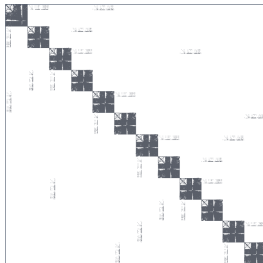
两张图都来自同一份导出的行列坐标，并统一采用“行号向下、列号向右”的矩阵显示方式。对角模式方向一致，说明差异来自绘图坐标约定而不是 CSR 结构本身。

数据来源：results/2026-05-16-mentor-action-items/sparse_pattern

同一矩阵重排序后可以呈现带状稀疏

Original CSR sparse pattern

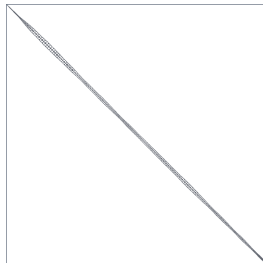
rev .inp node order, raster occupancy



原始 .inp 节点顺序：块状聚簇

RCM-reordered sparse pattern K(p,p)

reverse Cuthill-McKee, visualization only



$K(p,p)$: RCM 重排序后呈现带状

图 7 | mentor 图中 $B(p,p)$ 的对应解释

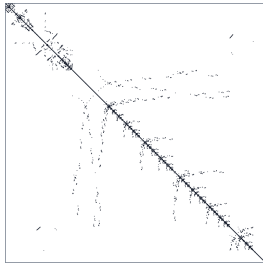
左图是不改变输入编号的原始 CSR pattern；右图只对可视化使用 reverse Cuthill–McKee 排列 p 。同一矩阵的非零数保持 27,502,200，带宽从 392,948 降到 9,314，说明块状图主要来自原始编号顺序而不是组装错误。

数据来源：assets/windhub_physics_tet4_visual_metadata.json；RCM 仅用于可视化

未压缩局部窗口：每个点都是真实行列坐标

Exact local sparse window

rows [0,4096], cols [0,4096], no binning

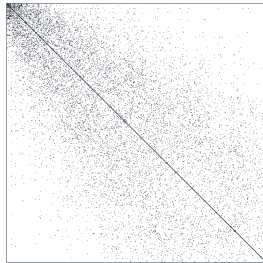


window size=78434, exact coordinates, no raster aggregation

固定窗口：rows/cols [0, 4096)

Auto-selected exact sparse window

rows [204800,208896], cols [204800,208896], no binning



window size=117838, exact coordinates, no raster aggregation

自动高密度窗口：rows/cols [204800, 208896)

图 8 | 全局图必须压缩，局部图可以不压缩

完整 WindHub 矩阵若按 1:1 像素显示需要约 685152^2 个像素，不适合作为图片。这里截取 **4096 x 4096** 的局部窗口，直接画真实 row,col 坐标，不做 binning 或 raster aggregation。

数据来源：assets/windhub_physics_tet4_visual_exact_window_*.csv/svg/png

CSR 三数组展示真实的组装后存储结构

行指针片段: row_offsets

	i	0	1	2	3
row_offsets[i]	0	27	54	81	
row_offsets[i+1]	27	54	81	102	
本行非零数	27	27	27	21	

解释
第 0 行的非零元存放在 p=0..26。这些位置同时索引 col_indices[p] 和 values[p]。

第 0 行前若干个三元项

p	row	col	value
0	0	0	1.328×10^{12}
1	0	1	2.892×10^{11}
2	0	2	-1.045×10^{11}
3	0	3	-8.727×10^{10}
4	0	4	2.972×10^{10}
5	0	5	7.323×10^{10}
6	0	45	-1.448×10^{11}
7	0	46	-1.881×10^{11}

图 9 | 组装后 CSR 三数组的局部窗口

这不是另一种数学矩阵，而是同一个整体刚度矩阵在内存里的确定存储方式：row_offsets 定位每一行的切片，col_indices 给出非零元列号，values 保存组装后的刚度值。

数据来源：assets/windhub_physics_tet4_serial_csr_window.csv；行窗口 [0,4)

五类数值后端都复用同一套 CSR 和 scatter

后端	它怎样写 <code>values[p]</code>	主要代价
<code>cpu_atomic</code>	多线程直接写共享 CSR，用原子加法保护每次累加。	热点位置会有 <code>atomic</code> 争用。
<code>cpu_lock_guard</code>	多线程直接写共享 CSR，每个非零元配一把锁。	<code>mutex</code> 内存和加锁/解锁开销大。
<code>cpu_private_csr</code>	每个线程写自己的 <code>values</code> 副本，最后逐项归并。	额外内存约为 <code>threads x nnz x sizeof(double)</code> 。
<code>cpu_graph_coloring</code>	先把互不冲突的单元分到同一颜色组，组内无锁并行写。	着色预处理和颜色组负载不均。
<code>cpu_row_owner</code>	按 CSR 行分 owner，每个线程只写自己拥有的行。	任务列表内存，以及单元可能被重复计算。

我的结论

五类后端的共同点是复用符号阶段的地址表；真正不同的是并发写入冲突如何处理。

数据来源：`numeric_assembly_algorithm_rehearsal.md`; `docs/cpu/cpu_algorithms.md`

内存：我先把三类指标拆开

程序能精确估算的数据结构

- 常驻符号结构：CSR 结构和写入计划。
- 输出矩阵：全局矩阵的数值数组。
- 临时缓冲：并行符号构建缓冲或直接组装贡献数组。

操作系统实际观察到的峰值

- 独立进程峰值常驻内存（isolated RSS）。
- 它会包含运行时、分配器和读写文件等额外开销。
- 所以我把估算字节和 RSS 分开汇报。

我的结论

估算字节解释“数据结构为什么占内存”；RSS 解释“这一条命令实际最高用到多少内存”。

无符号直接组装的 2.39 GiB 是临时贡献缓冲

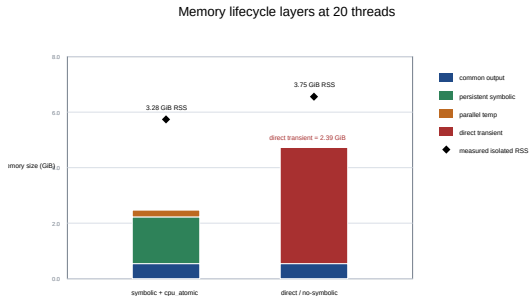


图 8 | 符号复用路线与无符号直接路线的内存生命周期

左柱是“符号结构 + 原子累加数值后端”，右柱是“无符号直接组装”。彩色堆叠表示程序可解释的数据结构，黑色菱形表示独立进程峰值 RSS。右柱中的 **2.39 GiB** 是直接组装先生成单元贡献、再排序归并所需的临时贡献缓冲。

数据来源：由 isolated_symbolic_memory.csv 派生；20 线程

已有内存热力图用于横向解释不同算法的内存形状

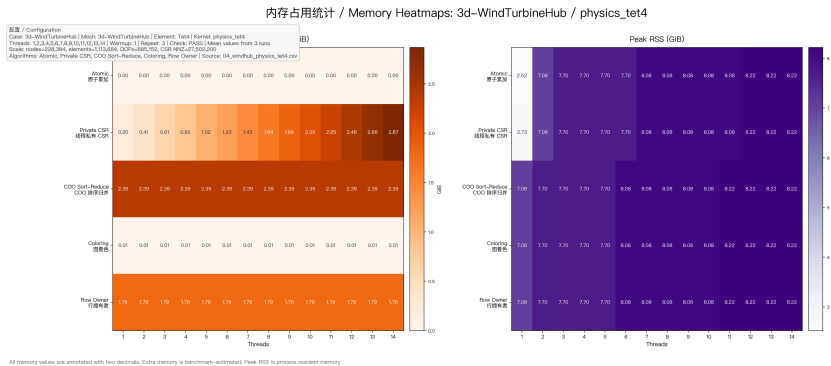


图 9 | WindHub 物理核函数的多算法内存热力图

这张图展示不同算法随线程数变化时的内存形状。它帮助我解释：有些算法用线程私有矩阵换同步开销，有些算法用锁数组保证写入安全，直接组装则主要消耗临时贡献缓冲。

数据来源：results/2026-04-28-12charts-repeat3-threads1to14/presentation_charts_12_v2

我为什么仍然把 `cpu_atomic` 作为主线数值后端

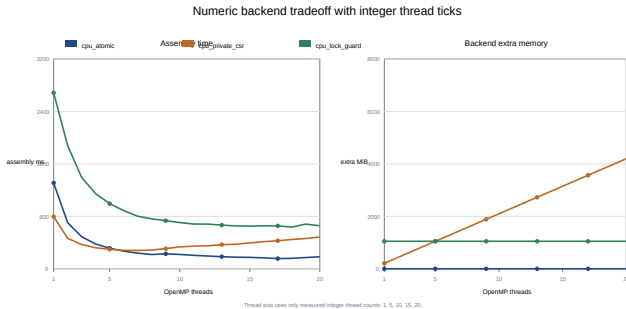


图 10 | 三类数值后端的速度和内存对照

左图纵轴是数值组装时间，越低越好；右图纵轴是数值后端额外内存。横轴只标出整数线程刻度。`cpu_atomic` 在 Intel 全核心结果中速度最好，并且额外后端内存为 0。

数据来源：[results/2026-05-20-linux-intel-symbolic-memory-full-host](#)

我会主动说明这次证据的边界

- 这组 Linux Intel 结果是单次全核心扫描，不是 3 次重复平均。
- 这次没有做 P/E-core 隔离，只回答整机 1 到 20 物理线程。
- 当前正确性参考矩阵来自 C++ `cpu_serial`，不是 Abaqus/MATLAB 外部矩阵。
- MATLAB 图只用于稀疏模式可视化，不用于证明数值矩阵来自外部正确参考。
- 估算峰值字节和独立进程 RSS 我会分开讲，不互相替代。

我的结论

这些边界不是削弱结论，而是说明我现在的证据链可复查、可继续补强。

我会按这些高风险追问提前演练

导师最可能追问

- 符号组装到底并行了哪一段？
- 数值组装怎样复用 CSR 和 scatter？
- 加速是不是来自数值后端，而不是符号阶段？
- 2.39 GiB 到底是总内存还是临时缓冲？

我会主动守住的边界

- 正确性参考是 C++ `cpu_serial`。
- MATLAB 只用于稀疏模式可视化。
- 这次是 single full sweep，不是 repeat 平均。
- 外部 Abaqus/MATLAB reference matrix 是下一步。

我的结论

我已经把完整自问自答和五类数值后端讲解整理成会前演练稿，正式汇报时只在问答环节展开。

数据来源：mentor_qna_rehearsal.md; numeric_assembly_algorithm_rehearsal.md

我最后会这样向导师复述

① 符号组装阶段已经并行化。

在同一数值后端内，20 线程的并行符号构建能把完整路径总时间缩短到串行符号构建的约四分之一到三分之一。

② 正确性每轮都有当前项目内参考。

我现在把所有候选矩阵都和 C++ `cpu_serial` 参考矩阵比较；外部 Abaqus/MATLAB 参考矩阵是下一步补强项。

③ 无符号直接组装不等于低内存。

它避免常驻符号结构，但在 20 线程下需要承担 **2.39 GiB** 临时贡献缓冲。

我的结论

我下一步最自然的补强方向，是导入外部参考矩阵，并把符号内存实验扩展成多次重复的最终证据。